



쿠버네티스 심화 실습

kubectl → Helm → Prometheus·Grafana, 하나로 이어지는 실습

3일차 오전

PART 1

Pod와 Deployment



Pod



Pod = 컨테이너를 감싸는 쿠버네티스의 가장 작은 배포 단위

하나의 Pod 안에 컨테이너가 하나 이상 들어가고, 같은 Pod 안의 컨테이너는 네트워크(IP)를 공유합니다.

Deployment



Deployment = 여러 개의 Pod를 원하는 개수만큼 유지해 주는 관리자

Pod가 죽으면 자동으로 다시 만들고, replica 수를 늘리거나 줄이고, 이미지를 새 버전으로 바꾸는 것까지 관리합니다.

실습 1. Pod에 리소스 제한 걸기

컨테이너가 쓸 수 있는 자원의 최대치를 정합니다

pod-resource.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: my-resource-pod
spec:
  containers:
  - name: app
    image: nginx:1.25
    resources:
      requests:
        memory: "64Mi"
      limits:
        memory: "128Mi"
```

핵심 requests는 최소 보장, limits는 최대 허용입니다 — limits를 넘기면 컨테이너가 강제 종료될 수 있습니다.

Pod 대신 Deployment를 쓰는 이유

직접 만든 Pod의 한계를 확인합니다

실험

`kubectl delete pod my-nginx` 로 방금 만든 Pod를 지워보세요.

결과

Pod가 사라진 뒤 다시 만들어지지 않습니다 — `kubectl get pod`에 더 이상 나타나지 않습니다.

문제

실제 서비스라면 이 순간부터 장애입니다. 누군가 지켜보다가 다시 만들어줘야 합니다.

해결

Deployment는 Pod가 사라지면 자동으로 다시 만들어 줍니다. 다음 실습에서 직접 확인합니다.

실습 2. Deployment 만들기

nginx Pod 3개를 Deployment로 관리합니다

`deploy-nginx.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: dep-nginx
  template:
    metadata:
      labels:
        app: dep-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

실습 3. Pod를 강제로 지워보기

Deployment의 자동 복구를 직접 눈으로 확인합니다

명령

```
# Pod 이름은 실습10에서 확인한 이름으로 교체
kubectl delete pod my-deploy-xxxxx
# 곧바로 다시 확인
kubectl get pod -w
```

확인 지운 Pod는 사라지지만, 새 이름의 Pod가 곧바로 다시 생깁니다 — replicas: 3을 계속 유지하기 때문입니다.

실습 4. replica 수 조정하기

명령 한 줄로 늘리고 줄입니다

명령

```
# 5개로 늘리기
kubectl scale deployment my-deploy --replicas=5
# 다시 2개로 줄이기
kubectl scale deployment my-deploy --replicas=2
```

확인 명령 실행 직후 `kubectl get pod`를 반복 실행해서 Pod 개수가 실제로 바뀌는 과정을 지켜보세요.

흔한 에러 1. ImagePullBackOff

일부러 재현하고, 원인을 찾아봅니다

재현

```
# 존재하지 않는 이미지 이름으로 일부러 실행  
kubectl run bad-pod --image=nginx:존재하지않는태그
```

원인 확인

```
kubectl describe pod bad-pod | tail -5
```

원인 이미지 이름이나 태그(버전)를 잘못 적었을 때 발생합니다. Events에 "Failed to pull image" 문구가 보입니다.

흔한 에러 2. CrashLoopBackOff

컨테이너가 계속 재시작을 반복하는 상황

재현

```
# 실행하자마자 바로 끝나는 명령으로 일부러 재현  
kubectl run bad-pod2 --image=busybox --command -- echo hi
```

상태 확인

```
kubectl get pod bad-pod2
```

원인 컨테이너 안의 프로세스가 바로 종료되면, 쿠버네티스는 계속 재시작을 시도합니다 — RESTARTS 숫자가 계속 올라갑니다.

흔한 에러 3. Pending 상태

Pod가 아예 시작도 못 하는 상황

재현

```
# 실제로 없는 만큼의 메모리를 요청해서 재현  
kubectl run bad-pod3 --image=nginx --requests=memory=999Gi
```

원인 확인

```
kubectl describe pod bad-pod3 | tail -5
```

원인 실습 환경(노드)에 그만큼의 자원이 없으면 Pod를 어디에도 배치하지 못해 Pending 상태로 남습니다.

문제가 생기면 이 순서로 확인하세요

오늘 만든 세 가지 에러 모두 같은 순서로 진단했습니다

1. get pod

STATUS 컬럼으로 어떤 종류의 문제인지 먼저 확인 (Pending / CrashLoopBackOff / ImagePullBackOff 등)

2. describe

Events 항목에서 구체적인 원인 메시지를 확인

3. logs

컨테이너가 실행은 됐지만 내부에서 에러가 난 경우, 로그로 원인 확인

자주 쓰는 kubectl 명령어 모음

오늘 실습한 명령을 한 장으로 정리합니다

분류	명령
조회	get pod / get deployment / get rs -o wide
진단	describe pod / logs / logs -f
접속	exec -it <pod> -- sh
운영	scale --replicas / set image
이력	rollout status / rollout history / rollout undo

Deployment가 실무에서 막아주는 상황

서버 한 대가 갑자기 죽는다면

Pod만 있었다면

새벽에 서버 한 대가 죽으면, 누군가 알아채고 수동으로 다시 만들 때까지 서비스가 멈춥니다.

Deployment였다면

몇 초 안에 자동으로 새 Pod가 생성되어, 사람이 잠든 사이에도 서비스가 계속 유지됩니다.

핵심 "장애 상황에서도 정해진 개수를 계속 유지한다"는 것이 Deployment를 쓰는 가장 큰 이유입니다.

정리 3. 네트워킹

Pod는 어떻게 서로, 그리고 외부와 통신할까



모든 Pod는 고유 IP를 가집니다

NAT 없이 Pod끼리 직접 통신합니다 (Flat Network).



Service가 고정된 진입점 역할을 합니다

Pod IP가 바뀌어도 Service의 IP·DNS 이름은 그대로입니다.



kube-proxy가 실제 연결을 담당합니다

Service로 온 요청을 실제 Pod IP로 연결(DNAT)해 줍니다.

PART 2

왜 Helm이 필요한가

kubectl만으로 배포할 때 무엇이 불편한지부터 확인합니다



kubectl만으로 배포할 때의 문제

01

Deployment, Service, ConfigMap을 각각 다른 파일로 따로 작성하고 관리해야 합니다.

02

환경(개발/운영)이 바뀔 때마다 YAML 안의 값을 사람이 직접 찾아서 고쳐야 합니다.

03

몇 번째 버전을 배포했는지, 문제가 생기면 어디로 되돌려야 하는지 기록이 남지 않습니다.

Helm이란?

쿠버네티스를 위한 '규격화된 배포 체크리스트' 도구



Helm = Kubernetes 애플리케이션을 위한 패키지 매니저

여러 개의 YAML 파일을 하나의 체크리스트(Chart)로 묶어서, 설치·업데이트·삭제를 한 번의 명령으로 처리하는 도구입니다.

- ✓ 우분투에서 프로그램을 apt-get install 하듯, Helm은 쿠버네티스 애플리케이션을 한 줄로 설치합니다.
- ✓ 체크리스트(Chart) 안의 내용만 바꾸면 되므로, YAML을 매번 새로 쓸 필요가 없습니다.
- ✓ 언제 무엇을 설치했는지 이력이 남아, 문제가 생기면 이전 상태로 쉽게 되돌릴 수 있습니다.

Helm의 3가지 구성 요소

이 세 단어만 정확히 구분하면 절반은 끝입니다



Chart

이사 체크리스트

애플리케이션 배포에 필요한 YAML
들을 묶어놓은 '설계도' 한 벌입니
다.



Repository

체크리스트 카탈로그

Chart를 모아두고 다운로드할 수
있는 저장소입니다. (인터넷 또는
사내 저장소)



Release

실제로 진행된 이사 건

Chart를 설치하면 만들어지는 결과
물입니다. 같은 Chart로 여러 번 설
치하면 Release가 여러 개 생깁니
다.

Part 2 정리

다음 파트로 넘어가기 전에 확인할 것

문제 YAML을 하나하나 따로 관리하면 파일도 많고, 환경별로 값을 고치기도 어렵고, 버전 기록도 남지 않습니다.

해결 Helm은 여러 YAML을 Chart라는 하나의 체크리스트로 묶어 관리합니다.

용어 Chart(체크리스트) · Repository(카탈로그) · Release(실제 설치 결과물) — 이 세 단어를 구분할 수 있으면 다음 파트로 넘어갈 준비가 된 것입니다.

PART 3

Chart 구조 이해하기

체크리스트(Char) 안에 실제로 무엇이 들어있는지 열어봅니다



Chart 안에 들어있는 것

체크리스트를 펼쳐서 안을 들여다봅니다



Chart는 폴더(디렉터리) 하나입니다

특별한 파일 형식이 아니라, 정해진 이름 규칙을 따르는 평범한 폴더입니다.

이 폴더 안에는 '이 패키지가 무엇인지'를 적은 파일, '사용자가 바꿀 수 있는 값'을 적은 파일, 그리고 '실제로 배포될 내용'을 담은 파일들이 함께 들어 있습니다.

다음 슬라이드에서 실제 구조를 하나씩 확인합니다.

Chart 디렉터리 구조

helm create로 만들면 이 구조가 자동으로 생성됩니다

mychart 폴더 구조

```
mychart/
├── Chart.yaml           # 이 패키지가 무엇인지 (이름, 버전, 설명)
├── values.yaml          # 사용자가 바꿀 수 있는 기본 설정값
├── charts/              # 이 Chart가 의존하는 다른 Chart
├── templates/           # 실제로 배포될 K8s YAML 템플릿
│   ├── deployment.yaml
│   ├── service.yaml
│   └── NOTES.txt        # 설치 후 안내 메시지
└── .helmignore          # 패키징 시 제외할 파일
```

확인 templates/ 안의 deployment.yaml, service.yaml은 2일차에 쓰던 것과 같은 형식입니다.

가장 먼저 봐야 할 파일 3가지

폴더 전체를 외울 필요는 없습니다



Chart.yaml

이 패키지가 무엇인지 — 이름, 버전, 설명



values.yaml

사용자가 바꿀 수 있는 설정값



templates/

실제로 배포될 K8s YAML 원본

핵심 charts/, .helmignore는 지금 몰라도 됩니다 — 이 3개 파일만 이해하면 Chart를 읽고 만들 수 있습니다.

실습: 나만의 Chart 만들기

명령어 한 줄이면 기본 구조가 통째로 생성됩니다

1. Chart 생성

```
helm create mychart
```

2. 생성된 구조 확인

```
tree mychart
mychart/
├── Chart.yaml
├── values.yaml
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    └── ...
```

Part 3 정리

다음 파트로 넘어가기 전에 확인할 것

구조 Chart는 폴더 하나이고, 그 안에 Chart.yaml · values.yaml · templates/가 핵심입니다.

생성 helm create 명령 한 줄로 기본 구조가 자동으로 만들어집니다.

다음 이제 values.yaml을 이용해서, 같은 Chart를 환경마다 다르게 배포하는 방법을 배웁니다.

PART 4

환경마다 다른 값 적용하기

체크리스트는 그대로 두고, 옵션값만 바꿔서 설치하는 방법을 배웁니다



개발 환경과 운영 환경, 다른 점이 많습니다

같은 애플리케이션인데 매번 YAML을 새로 써야 할까요?

	개발 환경	운영 환경
서버 대수	1대	5대
메모리	적게	크게
이미지 태그	latest (최신)	1.30 (고정 버전)

질문 템플릿(deployment.yaml)까지 통째로 다시 쓸 필요가 있을까요? 바뀌는 건 몇 개의 '값'뿐인데요.

해결책: 템플릿은 그대로, 값만 바꾼다

비유 — 이력서 양식과 작성 내용



templates/ — 이력서 양식

이름 칸, 학력 칸, 경력 칸의 위치와 형식은 누구나 똑같습니다.

쿠버네티스에서는 deployment.yaml, service.yaml의 구조가 이 '양식'에 해당합니다.



values.yaml — 실제로 채워 넣는 내용

이름, 학력, 경력처럼 사람마다 다른 부분만 따로 적어 넣습니다.

양식을 매번 새로 그릴 필요 없이 빈칸만 채우면 되는 것과 같습니다.

실습 1. 두 번째 Chart 만들기

httpd용 Chart를 새로 만들어 봅니다

명령

```
# 새 Chart 생성
helm create myhttpd
# 기본 이미지(nginx) 대신 httpd로 바꿔서 설치
helm install web-httpd ./myhttpd --set image.repository=httpd --set image.tag=2.4
```

확인 이전 Helm 세션에서는 nginx Chart만 다뤘습니다. 오늘은 다른 이미지로도 같은 과정을 반복해서 손에 익힙니다.

실습 2. values.yaml 수정 후 재설치

값을 바꾸고 upgrade로 반영합니다

명령

```
# values.yaml에서 replicaCount: 1 → 3 으로 수정 후
helm upgrade web-httpd ./myhttpd
# 반영 확인
kubectl get pod -l app.kubernetes.io/instance=web-httpd
```

핵심 install은 처음 한 번만, 그 이후 값을 바꿀 때는 upgrade를 씁니다.

실습 3. --set 연습 A

replicaCount만 바꿔서 설치해 봅니다

명령

```
# replica 4개로 설치  
helm upgrade web-httpd ./myhttpd --set replicaCount=4
```

확인

```
kubectl get pod -l app.kubernetes.io/instance=web-httpd
```

실습 4. --set 연습 B

여러 값을 한 번에 바꿔봅니다

명령

```
# replica 수와 서비스 타입을 동시에 변경
helm upgrade web-httpd ./myhttpd \
  --set replicaCount=2 \
  --set service.type=NodePort
```

확인 kubectl get svc로 service.type이 NodePort로 바뀌었는지 확인하세요.

실습 5. -f로 개발 환경 설치하기

값 파일을 새로 만들어 설치합니다

파일 작성

```
# values-dev.yaml
replicaCount: 1
resources:
  limits:
    memory: "128Mi"
```

적용

```
helm upgrade web-httpd ./myhttpd -f values-dev.yaml
```

실습 6. -f로 운영 환경 설치하기

dev와 비교하며 prod 파일도 만들어 봅니다

	values-dev.yaml	values-prod.yaml
replicaCount	1	3
memory limit	128Mi	512Mi

적용

```
helm upgrade web-httpd ./myhttpd -f values-prod.yaml
```

실습 7. 값 파일 두 개를 겹쳐서 쓰기

실무에서 자주 쓰는 패턴입니다

명령

```
# base 값 위에 prod 값을 덮어씁니다 (뒤에 오는 파일이 우선)
helm upgrade web-httpd ./myhttpd \
  -f values-base.yaml \
  -f values-prod.yaml
```

핵심 -f를 여러 번 쓰면 뒤에 오는 파일의 값이 앞의 값을 덮어씁니다. 공통 값은 base에, 환경별로 다른 값만 별도 파일에 두는 방식입니다.

--set vs -f, 선택 기준

정답은 하나가 아니라 '상황에 맞게'입니다

	--set	-f
방식	명령어에 직접 값 입력	별도 파일로 값 관리
적합한 경우	간단한 테스트, 값 1~3개	값이 많거나 환경을 나눌 때
재사용/협업	어려움	파일로 공유하기 쉬움

참고 두 옵션은 동시에 쓸 수도 있습니다 — 이 경우 --set이 우선 적용됩니다.

Part 4 정리

다음 파트로 넘어가기 전에 확인할 것

원리 templates/(양식)는 그대로 두고, values.yaml(내용)만 바꿔서 환경별로 배포합니다.

방법 값이 적으면 --set, 값이 많거나 파일로 관리하고 싶으면 -f를 씁니다.

다음 이제 설치한 Chart를 업그레이드하고, 문제가 생기면 되돌리는 방법을 배웁니다.

PART 5

설치부터 롤백까지

Chart를 설치한 뒤 실제로 운영하면서 겪는 네 가지 동작을 다룹니다



Helm 생명주기 네 단계

이 순서로 기억하면 됩니다

install

처음 설치합니다. 같은 이름이 이미 있으면 에러가 납니다.

upgrade

이미 설치된 것의 값이나 Chart 내용을 변경합니다.

rollback

설치는 유지한 채, 이전 버전 상태로 되돌립니다.

uninstall

릴리즈를 완전히 삭제합니다 — 모든 리소스가 제거됩니다.

주의 uninstall과 rollback을 혼동하지 마세요 — 실수로 잘못 업그레이드했을 때는 uninstall이 아니라 rollback을 씁니다.

실습 8. 같은 Chart로 Release 두 개 만들기

Chart 하나, Release는 여러 개일 수 있습니다

명령

```
# 이름만 다르게 두 번 설치
helm install web-a ./myhttpd
helm install web-b ./myhttpd
```

확인 kubectl get pod로 web-a, web-b 각각의 Pod가 따로 떠 있는지 확인하세요 — 서로 완전히 독립적인 Release입니다.

실습 9. 설치된 Release 한눈에 보기

helm list로 전체를 확인합니다

명령

```
helm list
```

NAME	NAMESPACE	REVISION	STATUS	CHART
web-a	default	1	deployed	myhttpd-0.1.0
web-b	default	1	deployed	myhttpd-0.1.0
web-httpd	default	7	deployed	myhttpd-0.1.0

확인 CHART 컬럼은 모두 같은데 NAME과 REVISION은 다릅니다 — Release가 서로 독립적으로 관리된다는 뜻입니다.

실습 10. upgrade 반복해서 걸어보기

값을 여러 번 바꾸며 리비전을 쌓습니다

명령

```
# 세 번 연달아 값을 바꿔서 업그레이드
helm upgrade web-a ./myhttpd --set replicaCount=2
helm upgrade web-a ./myhttpd --set replicaCount=3
helm upgrade web-a ./myhttpd --set service.type=NodePort
```

다음 이렇게 쌓인 이력을 다음 실습에서 helm history로 확인합니다.

실습 11. 전체 변경 이력 확인하기

방금 쌓은 리비전을 확인합니다

명령

```
helm history web-a
```

REVISION	STATUS	CHART	DESCRIPTION
1	superseded	myhttpd-0.1.0	Install complete
2	superseded	myhttpd-0.1.0	Upgrade complete
3	superseded	myhttpd-0.1.0	Upgrade complete
4	deployed	myhttpd-0.1.0	Upgrade complete

실습 12. 특정 리비전으로 롤백하기

REVISION 1(최초 설치 상태)로 되돌립니다

명령

```
# 리비전 번호를 직접 지정해서 롤백  
helm rollback web-a 1
```

결과 확인

```
helm history web-a
```

확인 history를 다시 보면 REVISION 5가 새로 생기고, 그 내용은 REVISION 1과 같습니다 — 롤백도 하나의 새 변경으로 기록됩니다.

실습 13. 현재 상태 한 번에 확인하기

helm status로 요약 정보를 봅니다

명령

```
helm status web-a
```

확인 STATUS, REVISION, 그리고 Chart가 설치 후 안내한 NOTES.txt 내용까지 한 번에 볼 수 있습니다.

실습 14. Chart의 기본값 확인하기

values.yaml을 열지 않고도 기본값을 볼 수 있습니다

명령

```
helm show values ./myhttpd
```

쓰임새 다른 사람이 만든 Chart를 처음 쓸 때, 어떤 값을 --set이나 -f로 바꿀 수 있는지 미리 확인하는 용도로 유용합니다.

흔한 실수 1. YAML 들여쓰기 오류

일부러 오류를 만들고 lint로 찾아봅니다

재현 및 확인

```
# values.yaml에서 일부러 들여쓰기를 한 칸 어긋나게 수정 후
helm lint ./myhttpd
Error: values.yaml: error converting YAML to JSON...
```

원인 YAML은 들여쓰기(공백 개수)로 구조가 정해집니다 — 탭(Tab)을 쓰거나 공백 수가 안 맞으면 이런 오류가 납니다.

설치 전 문법 체크하기

실제로 설치하지 않고 먼저 확인하는 세 가지 방법

1. 문법 검사

```
# 문법 오류가 있는지 검사  
helm lint mychart
```

2. 결과 미리보기

```
# 실제로 어떤 YAML이 만들어지는지 미리보기  
helm template mychart
```

3. 설치 시뮬레이션

```
# 실제 설치 없이 시뮬레이션만 해보기  
helm install my-release mychart --dry-run --debug
```

비유 이사 전에 체크리스트를 한 번 더 검토하고, 실제로 짐을 싸기 전에 배치도만 그려보는 것과 같습니다.

흔한 실수 3. 이름이 이미 있다는 에러

install은 같은 이름으로 두 번 쓸 수 없습니다

재현

```
# 이미 설치된 이름으로 다시 install 시도
helm install web-a ./myhttpd
Error: INSTALLATION FAILED: cannot re-use a name...
```

해결 값을 바꿔서 다시 배포하고 싶다면 install이 아니라 upgrade를 씁니다. 이름을 다르게 하고 싶다면 install에 새 이름을 씁니다.

Helm 문제가 생기면 이 순서로 확인하세요

오늘 만든 세 가지 실수 모두 이 순서로 해결했습니다

1. lint

`helm lint ./chart` — 문법 오류부터 확인

2. template

`helm template ./chart` — 실제로 어떤 YAML이 만들어지는지 미리보기

3. --dry-run

`helm install ... --dry-run --debug` — 설치 없이 시뮬레이션

4. status/history

이미 설치된 것이라면 `helm status`, `helm history`로 현재 상태 확인

자주 쓰는 Helm 명령어 모음

오늘 실습한 명령을 한 장으로 정리합니다

분류	명령
생성/검증	helm create / helm lint / helm template
설치/변경	helm install / helm upgrade --set / helm upgrade -f
조회	helm list / helm status / helm show values
이력/복구	helm history / helm rollback
삭제	helm uninstall

핵심 정리

Helm 기본기 정리

- 1 Helm은 여러 YAML을 Chart라는 하나의 체크리스트로 묶어서 관리하는 쿠버네티스 패키지 매니저입니다.
- 2 Chart(체크리스트) · Repository(카탈로그) · Release(실제 설치 결과물), 세 용어를 구분할 수 있어야 합니다.
- 3 Chart는 폴더 하나이며, Chart.yaml · values.yaml · templates/ 세 가지가 핵심입니다.
- 4 templates/(양식)는 그대로 두고, values.yaml(내용)만 바꿔서 환경별로 배포합니다.
- 5 값이 적으면 --set, 많으면 -f, 여러 -f를 겹쳐 쓰면 base 값 위에 환경별 값을 덮어쓸 수 있습니다.

핵심 정리

Helm 심화·실전 정리

- 1 같은 Chart로 여러 개의 독립된 Release를 동시에 운영할 수 있습니다 — 이름만 다르면 됩니다.
- 2 `install(설치) → upgrade(변경) → rollback(되돌리기) → uninstall(삭제)`, 네 동작은 서로 다른 상황에 씁니다.
- 3 `helm rollback <release> <리비전 번호>`로 특정 시점 상태까지 정확히 되돌릴 수 있습니다.
- 4 `helm show values`로 Chart의 기본값을, `helm status·history`로 현재 상태와 전체 이력을 확인할 수 있습니다.
- 5 실수로 잘못 배포했을 때는 `uninstall`이 아니라 `rollback`을 먼저 시도합니다.
- 6 설치 전에는 `helm lint · helm template · --dry-run` 순서로 미리 확인하는 습관을 들입니다.

PART 1

왜 모니터링이 필요한가

지금까지 써온 명령의 한계부터 확인합니다



지금까지는 눈으로만 확인했습니다

kubectl get pod로는 '떠 있는지'만 알 수 있습니다

01 kubectl get pod로는 Pod가 살아있는지만 알 수 있고, CPU·메모리를 얼마나 쓰는지 알 수 없습니다.

02 Pod가 느려지거나 자꾸 재시작되는데, 정확히 무엇이 원인인지 숫자로 확인할 방법이 없었습니다.

03 10분 전, 1시간 전에 어떤 상태였는지는 이미 지나가 버려서 알 수 없습니다.

질문 "이 Pod가 지금 CPU를 얼마나 쓰고 있지?"에 답하려면 무엇이 필요할까요?

실습 1. kubectl top으로 사용량 보기

가장 간단한 방법부터 확인합니다

명령

```
# 노드/파드별 CPU·메모리 사용량 확인
kubectl top node
kubectl top pod
```

안 되면 "error: Metrics API not available" 가 나오면 metrics-server가 꺼져 있는 것입니다 — 다음 슬라이드에서
봅니다.

kubectl top의 정체: metrics-server

클러스터의 '지금 이 순간' 체온계



metrics-server = 지금 이 순간의 CPU·메모리 사용량만 알려주는 도구

체온계로 지금 체온을 재는 것과 같습니다 — 방금 쎈 값 하나만 보여주고, 10분 전 체온이 몇 도였는지는 기억하지 못합니다.

kubectl top이 동작하려면 클러스터에 metrics-server가 켜져 있어야 합니다.

실습 환경에서 켜기

```
minikube addons enable metrics-server
```

과거 기록은 확인할 수 없습니다

metrics-server만으로는 부족한 이유



"10분 전에 CPU 사용량이 갑자기 튀었어요" — 확인할 방법이 없습니다

kubectl top은 지금 이 순간의 값만 보여주고 저장하지 않습니다. 화면을 계속 보고 있지 않았다면, 방금 지나간 순간은 다시 볼 수 없습니다.

문제를 진단하려면 시간에 따라 쌓인 기록이 필요합니다.

Prometheus란?

주기적으로 값을 재고, 시간과 함께 저장하는 시스템

- ✓ 정해진 주기(예: 15초)마다 클러스터의 여러 지표를 자동으로 수집합니다.
- ✓ 수집한 값을 시간 순서대로 저장합니다 — 이것을 시계열(time series) 데이터라고 부릅니다.
- ✓ "지난 1시간 동안 CPU가 어떻게 변했는지" 같은 질문에 답할 수 있습니다.

Part 1 정리

다음 파트로 넘어가기 전에 확인할 것

한계 `kubectl top(metrics-server)`은 지금 이 순간의 값만 보여주고, 기록을 남기지 않습니다.

해결 Prometheus는 주기적으로 값을 수집해 시간과 함께 저장합니다 — 과거 기록까지 확인할 수 있습니다.

다음 이제 Prometheus를 직접 설치하고, 값이 잘 쌓이는지 확인합니다.

PART 2

Prometheus 설치하고 확인하기

이미 배운 Helm으로 직접 설치합니다



Helm으로 설치하는 이유

Prometheus는 YAML이 수십 개짜리 애플리케이션입니다



Prometheus + Grafana + Alertmanager, 이 모든 걸 관리하는 Chart가 있습니다

직접 YAML을 하나하나 작성하면 수십 개가 필요하지만, prometheus-community라는 공개 Repository에 이미 만들어진 Chart(kube-prometheus-stack)가 있습니다.

지난 시간에 배운 helm install 한 줄로 전체를 설치합니다.

실습 2. kube-prometheus-stack 설치하기

공개 Repository를 추가하고 설치합니다

명령

```
# Repository 추가 (처음 한 번만)
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
# 설치 (namespace를 새로 만들면서 설치)
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

참고 helm repo add는 오늘 처음 나온 명령입니다 — 우리가 직접 만들지 않은, 남이 만든 Chart를 가져올 때 씁니다.
저장된 Chart 목록의 캐시

```
cat ~/.cache/helm/repository/
```

저장된 매핑표

```
cat ~/.config/helm/repositories.yaml
```

실습 3. 설치 확인하기

monitoring 네임스페이스 안을 들여다봅니다

명령

```
# monitoring 네임스페이스의 Pod들 확인
kubectl get pod -n monitoring
helm list -n monitoring
```

확인 Prometheus, Grafana, Alertmanager Pod가 모두 Running 상태가 될 때까지 1~2분 기다려주세요.

실습 4. Prometheus 화면 접속하기

port-forward로 브라우저에서 열어봅니다

명령

```
# 로컬 9090 포트로 연결
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus --address 0.0.0.0 9090:9090
```

접속 브라우저에서 `http://IP:9090` 으로 접속하세요. 이 터미널 창은 접속해 있는 동안 계속 열어두어야 합니다.

Prometheus가 측정하는 대상: Target

주기적으로 값을 가져오는 곳을 Target이라 부릅니다



Target = Prometheus가 주기적으로 값을 가져오는 대상

노드, kubelet, Pod 등 클러스터 안의 여러 구성요소가 각자 자신의 상태를 숫자로 내보내고 있습니다.

Prometheus는 이 대상들을 목록(Target)으로 등록해 두고, 주기적으로 찾아가서 값을 가져옵니다.

실습 5. Targets 화면 확인하기

Prometheus가 무엇을 측정하고 있는지 직접 봅니다



Status → Targets 메뉴로 이동하세요

화면에 여러 줄의 Target 목록이 보입니다. 상태(State)가 초록색 UP으로 표시된 항목이 정상적으로 값을 수집하고 있는 것입니다.

지금은 목록만 확인하고, 각 항목이 정확히 무엇인지는 외우지 않아도 됩니다.

값을 꺼내 보는 언어: PromQL

지금은 문법을 외우지 않고, 딱 두 가지만 기억합니다



지표 이름으로 조회

container_cpu_usage_seconds_total 처럼 지표 이름만 입력해도 값이 나옵니다.



{ } 안에 조건 추가

{namespace="default"} 처럼 중괄호 안에 조건을 넣어 원하는 것만 걸러냅니다.

참고 date가 다르면 결과가 안 나올 수 있음

```
sudo timedatectl set-ntp false
sudo timedatectl set-ntp true
sudo systemctl restart systemd-timesyncd
timedatectl status
```

실습 6. CPU 사용량 조회하기

Graph 탭에서 직접 입력해 봅니다

PromQL

```
# 쿼리 입력창에 붙여넣기
container_cpu_usage_seconds_total{namespace="default"}
```

확인 Execute를 누르고 Graph 탭을 클릭하면, 시간에 따라 CPU 사용량이 그려진 선 그래프가 보입니다.

실습 7. 메모리 사용량 조회하기

이번엔 다른 지표로 같은 방식을 반복합니다

PromQL

```
# 쿼리 입력창에 붙여넣기
container_memory_usage_bytes{namespace="default"}
```

핵심 지표 이름만 다를 뿐, 조회하는 방식(중괄호 안에 조건 넣기)은 실습6과 똑같습니다.

흔한 문제. 그래프에 값이 표시되지 않는 경우

당황하지 말고 이 순서로 확인합니다

1. 시간 범위

화면 오른쪽 위의 시간 범위가 너무 좁지 않은지 확인 (1h 이상으로 넓혀보기)

2. namespace

쿼리의 namespace="default" 부분이 실제로 Pod가 있는 namespace와 맞는지 확인

3. Target 상태

Status → Targets에서 해당 항목이 UP인지 다시 확인

Part 2 정리

다음 파트로 넘어가기 전에 확인할 것

설치 Prometheus는 Helm으로 설치했습니다 — `helm repo add`로 남이 만든 Chart를 가져왔습니다.

동작 Target을 주기적으로 찾아가 값을 수집하고, 시간과 함께 저장합니다.

다음 이 값을 매번 쿼리로 찾지 않고, 화면에 그래프로 자동으로 보여주는 Grafana로 넘어갑니다.

PART 3

Grafana로 시각화하기

쿼리를 입력하지 않아도 그래프가 항상 떠 있는 화면을 만듭니다



Grafana란?



Grafana = Prometheus가 모은 데이터를 그래프로 보여주는 화면

데이터를 직접 수집하지 않고, Prometheus에 저장된 데이터를 가져와서 보여주기만 합니다. kube-prometheus-stack을 설치할 때 이미 함께 설치되었습니다.

실습 8. Grafana 접속하기

port-forward로 접속하고 로그인합니다

명령 및 로그인 정보

```
#Grafana 패스워드 확인
kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
# 새 터미널 창에서 실행 (9090은 이미 사용 중)
kubectl port-forward -n monitoring svc/monitoring-grafana --address 0.0.0.0 3000:80
# 기본 로그인 정보
아이디: admin
비밀번호: 확인한 패스워드
```

접속 브라우저에서 `http://IP:3000` 으로 접속하세요.

실습 9. 기본 제공 대시보드 둘러보기

이미 만들어진 대시보드가 여러 개 있습니다



왼쪽 메뉴 Dashboards로 이동하세요

"Kubernetes / Compute Resources / Namespace (Pods)" 같은 이름의 대시보드를 열어보세요.

이미 CPU·메모리 그래프가 그려져 있는 것을 확인할 수 있습니다 — 아무것도 만들지 않았는데도 Prometheus 데이터를 가져와 자동으로 그려줍니다.

minikube 같은 단일 노드/ 단일 클러스터 환경에서는 아무것도 나오지 않는 경우가 생김.

실습 10. 대시보드에서 내 Pod 찾기

지난 시간에 만든 Pod가 그래프에 보입니다



Namespace(Pods) 대시보드에서 default 네임스페이스를 선택하세요

화면 위쪽의 namespace 드롭다운에서 default를 고르면, 지난 시간에 만들었던 Pod들이 목록에 나타납니다.

내가 직접 만든 Pod의 CPU·메모리 사용량이 실제로 그래프에 찍혀 있는 것을 확인하세요.

실습 11. 나만의 패널 만들기 — CPU

새 대시보드에 패널을 하나 추가합니다

- 1 왼쪽 메뉴 Dashboards → New → New Dashboard
- 2 +모양 클릭 - Configure visualization 클릭 - Data source(Prometheus 선택)
- 3 Code 선택 후 쿼리 입력창에 실습6에서 썼던 CPU 쿼리를 붙여넣기
- 4 패널 제목을 "내 Pod CPU 사용량"으로 변경 후 Apply

실습 12. 대시보드 저장하기



화면 오른쪽 위 Save dashboard 클릭

대시보드 이름을 "내 실습 대시보드"처럼 알아보기 쉽게 정하고 저장하세요.

저장한 대시보드는 왼쪽 메뉴 Dashboards 목록에서 언제든지 다시 열어볼 수 있습니다.

실습 13. 부하를 줘서 그래프 움직임 보기

그래프가 실시간으로 반응하는 것을 확인합니다

명령

```
# CPU에 부하를 주는 Pod 실행
kubectl run stress --image=polinux/stress --command -- stress --cpu 2
# 30초 정도 기다린 뒤 정리
kubectl delete pod stress
```

확인 Grafana 대시보드를 새로고침하면서 CPU 그래프가 위로 솟구치는 것을 실시간으로 지켜보세요.